

MAKALAH
“TUGAS 6 SISTEM OPRASI”

Disusun Untuk Memenuhi Tugas Mata Kuliah
Sistem Oprasi

Dosen : Triawan Adi Cahyanto, M.Kom



Disusun Oleh:
Nama: Ragil Anggara Kasih Budiarto

NIM:2410651045

Prodi:Teknik Informatika

Kelas: TI – C

UNIVERSITAS MUHAMMADIYAH JEMBER

2025-2026

BAB I ANALISIS DEADLOCK

Jalankan program `deadlock_simple.c` dan jelaskan:

1. Mengapa program mengalami deadlock?

Program Mengalami deadlock karena kedua thread saling menunggu resource yang sedang di pegang thread lain. Contohnya: Thread 1 mengunci Lock 1, lalu ingin mengambil Lock 2 (yang sedang dipegang Thread 2). Thread 2 mengunci Lock 2, lalu ingin mengambil Lock 1 (yang sedang dipegang Thread 1). Jadi keduanya akan menunggu satu sama lain tanpa pernah melepas lock sehingga program akan berhenti total.

2. Kondisi deadlock apa saja yang terpenuhi?

- Mutual Exclusion
Setiap lock hanya bisa dipakai 1 thread dalam 1 waktu
- Hold and Wait
Thread 1 memegang lock 1 sambil menunggu lock 2 dan thread 2 melakukan sebaliknya.
- No Preemption
Lock tidak bisa direbut paksa jadi hanya bisa dilepas oleh thread yang memasangnya.
- Circular Wait
Thread 1 menunggu resource milik thread 2, dan thread 2 menunggu resource milik thread 1, sehingga berbentuk lingkaran.

3. Gambar diagram resource allocation graph-nya

Thread 1 → Lock 1 → Thread 2 → Lock 2 → Thread 1 (Terus Berulang)

atau dalam bentuk hubungan:

- T1 memegang Lock1
- T1 menunggu Lock2
- T2 memegang Lock2
- T2 menunggu Lock1

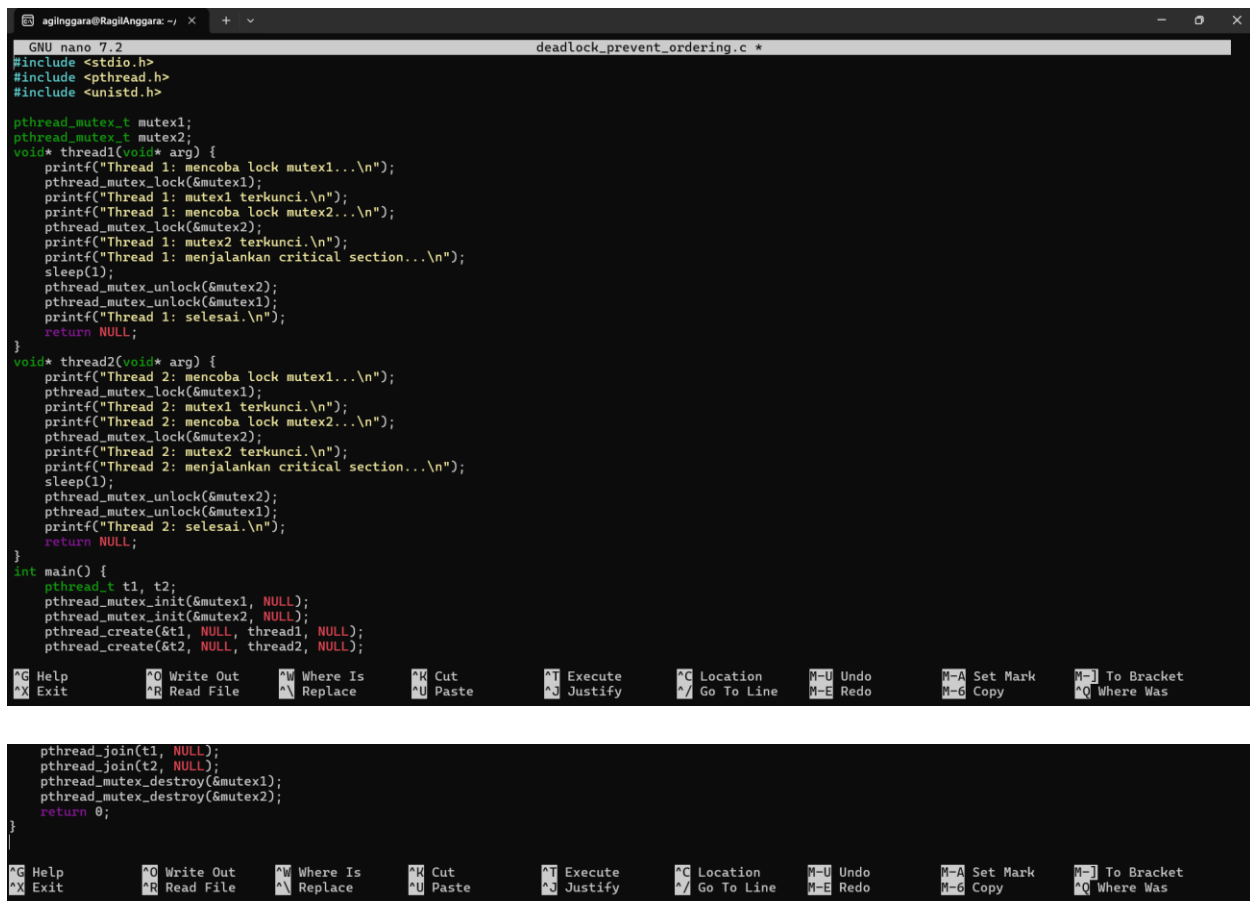
Ini membentuk cycle, yang merupakan indikator deadlock.

BAB II IMPLEMENTASI PENCEGAHAN

Modifikasi program `deadlock_simple.c` menggunakan teknik:

1. Lock ordering
2. Try-lock dengan retry
3. Bandingkan kedua metode tersebut

2.1 Lock Ordering



```
GNU nano 7.2 deadlock_prevent_ordering.c *
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t mutex1;
pthread_mutex_t mutex2;
void* thread1(void* arg) {
    printf("Thread 1: mencoba lock mutex1...\n");
    pthread_mutex_lock(&mutex1);
    printf("Thread 1: mutex1 terkunci.\n");
    printf("Thread 1: mencoba lock mutex2...\n");
    pthread_mutex_lock(&mutex2);
    printf("Thread 1: mutex2 terkunci.\n");
    printf("Thread 1: menjalankan critical section...\n");
    sleep(1);
    pthread_mutex_unlock(&mutex2);
    pthread_mutex_unlock(&mutex1);
    printf("Thread 1: selesai.\n");
    return NULL;
}
void* thread2(void* arg) {
    printf("Thread 2: mencoba lock mutex1...\n");
    pthread_mutex_lock(&mutex1);
    printf("Thread 2: mutex1 terkunci.\n");
    printf("Thread 2: mencoba lock mutex2...\n");
    pthread_mutex_lock(&mutex2);
    printf("Thread 2: mutex2 terkunci.\n");
    printf("Thread 2: menjalankan critical section...\n");
    sleep(1);
    pthread_mutex_unlock(&mutex2);
    pthread_mutex_unlock(&mutex1);
    printf("Thread 2: selesai.\n");
    return NULL;
}
int main() {
    pthread_t t1, t2;
    pthread_mutex_init(&mutex1, NULL);
    pthread_mutex_init(&mutex2, NULL);
    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    pthread_mutex_destroy(&mutex1);
    pthread_mutex_destroy(&mutex2);
    return 0;
}
```

Outputnya:

```

agilnggara@RagilAnggara:~/tugas6/deadlock_prevent_ordering$ gcc deadlock_prevent_ordering.c -o deadlock_prevent_ordering
agilnggara@RagilAnggara:~/tugas6/deadlock_prevent_ordering$ ./deadlock_prevent_ordering
Thread 1: mencoba lock mutex1...
Thread 1: mutex1 terkunci.
Thread 1: mencoba lock mutex2...
Thread 1: mutex2 terkunci.
Thread 1: menjalankan critical section...
Thread 2: mencoba lock mutex1...
Thread 1: selesai.
Thread 2: mutex1 terkunci.
Thread 2: mencoba lock mutex2...
Thread 2: mutex2 terkunci.
Thread 2: menjalankan critical section...
Thread 2: selesai.
agilnggara@RagilAnggara:~/tugas6/deadlock_prevent_ordering$ |

```

2.2 Try Lock dengan Retry

```

GNU nano 7.2 deadlock_prevent_trylock.c *
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
pthread_mutex_t mutex1;
pthread_mutex_t mutex2;
void* thread1(void* arg) {
    while (1) {
        printf("Thread 1: mencoba lock mutex1...\n");
        pthread_mutex_lock(&mutex1);
        printf("Thread 1: mencoba lock mutex2...\n");
        if (pthread_mutex_trylock(&mutex2) == 0) {
            // berhasil
            printf("Thread 1: semua mutex terkunci.\n");
            break;
        } else {
            // gagal + lepaskan dan coba lagi
            printf("Thread 1: gagal lock mutex2, retry...\n");
            pthread_mutex_unlock(&mutex1);
            usleep(100000); // 0.1 detik
        }
    }
    printf("Thread 1: critical section.\n");
    sleep(1);
    pthread_mutex_unlock(&mutex2);
    pthread_mutex_unlock(&mutex1);
    printf("Thread 1: selesai.\n");
    return NULL;
}
void* thread2(void* arg) {
    while (1) {
        printf("Thread 2: mencoba lock mutex2...\n");
        pthread_mutex_lock(&mutex2);
        printf("Thread 2: mencoba lock mutex1...\n");
        if (pthread_mutex_trylock(&mutex1) == 0) {
            printf("Thread 2: semua mutex terkunci.\n");
            break;
        } else {
            printf("Thread 2: gagal lock mutex1, retry...\n");
            pthread_mutex_unlock(&mutex2);
            usleep(100000);
        }
    }
    printf("Thread 2: critical section.\n");
    sleep(1);
    pthread_mutex_unlock(&mutex1);
    pthread_mutex_unlock(&mutex2);
    printf("Thread 2: selesai.\n");
    return NULL;
}
int main() {
    pthread_t t1, t2;
    pthread_mutex_init(&mutex1, NULL);
    pthread_mutex_init(&mutex2, NULL);
    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    pthread_mutex_destroy(&mutex1);
    pthread_mutex_destroy(&mutex2);
    return 0;
}

```

Output:

```

agilnggara@RagilAnggara:~/tugas6/deadlock_prevent_trylock$ gcc deadlock_prevent_trylock.c -o deadlock_prevent_trylock
agilnggara@RagilAnggara:~/tugas6/deadlock_prevent_trylock$ ./deadlock_prevent_trylock
Thread 1: mencoba lock mutex1...
Thread 1: mencoba lock mutex2...
Thread 1: semua mutex terkunci.
Thread 1: critical section.
Thread 2: mencoba lock mutex2...
Thread 1: selesai.
Thread 2: mencoba lock mutex1...
Thread 2: semua mutex terkunci.
Thread 2: critical section.
Thread 2: selesai.
agilnggara@RagilAnggara:~/tugas6/deadlock_prevent_trylock$ |

```

2.3 Perbandingan

Lock Ordering

Kelebihannya adalah efisien, simple, dan tidak ada retry, sedangkan kekurangannya adalah butuh desain urutan lock yang konsisten di seluruh program.

Try_Lock dan Retry

Kelebihannya adalah fleksibel, tidak harus sama dengan urutan lock nya, sedangkan kekurangannya adalah bisa banyak loop retry jadi tidak efisien.

BAB III

BANKER'S ALGORITHM

Buat Program baru dengan data:

- 4 Proses
- 3 jenis resource
- Available: [5, 4, 3]
- Implementasikan request resource dan cek keamanan sistem

Buat file bankers.c

```
GNU nano 7.2 bankers.c *
#include <stdio.h>
#include <stdbool.h>

#define P 4 // jumlah proses
#define R 3 // jumlah jenis resource

int available[R] = {5, 4, 3};
int max[P][R] = {
    {3, 2, 2},
    {1, 2, 3},
    {1, 3, 5},
    {2, 1, 4}
};
int allocation[P][R] = {
    {1, 0, 0},
    {0, 1, 2},
    {1, 1, 3},
    {0, 0, 2}
};
int need[P][R];
void calculateNeed() {
    for (int i = 0; i < P; i++) {
        for (int j = 0; j < R; j++) {
            need[i][j] = max[i][j] - allocation[i][j];
        }
    }
}
bool isSafe() {
    int work[R];
    bool finish[P] = {false};
    for (int i = 0; i < R; i++) {
        work[i] = available[i];
    }
    int count = 0;
    while (count < P) {
        bool found = false;
        for (int i = 0; i < P; i++) {
            if (!finish[i]) {
                bool canRun = true;
                for (int j = 0; j < R; j++) {
                    if (need[i][j] > work[j]) {

```

```

GNU nano 7.2                                bankers.c *
        canRun = false;
        break;
    }
    if (canRun) {
        for (int j = 0; j < R; j++) {
            work[j] += allocation[i][j];
        }
        finish[i] = true;
        found = true;
        count++;
    }
}
if (!found)
    return false; // tidak ada proses yang bisa jalan
return true;
}
bool requestResource(int p, int req[]) {
    for (int j = 0; j < R; j++) {
        if (req[j] > need[p][j]) {
            printf("ERROR: Request melebihi need proses!\n");
            return false;
        }
        if (req[j] > available[j]) {
            printf("ERROR: Resource tidak cukup tersedia!\n");
            return false;
        }
    }
    // coba alokasikan sementara (pretend)
    for (int j = 0; j < R; j++) {
        available[j] -= req[j];
        allocation[p][j] += req[j];
        need[p][j] -= req[j];
    }
    // cek aman?
    if (isSafe()) {
        printf("Request dapat diberikan. Sistem tetap aman.\n");
        return true;
    }
}
} else {
    // rollback
    for (int j = 0; j < R; j++) {
        available[j] += req[j];
        allocation[p][j] -= req[j];
        need[p][j] += req[j];
    }
    printf("Request TIDAK dapat diberikan (unsafe).\n");
    return false;
}
}
int main() {
    calculateNeed();
    printf("=== Banker's Algorithm ===\n");
    printf("Available awal: [%d, %d, %d]\n", available[0], available[1], available[2]);
    int p;
    int req[0];
    printf("\nMasukkan nomor proses (0-3): ");
    scanf("%d", &p);
    printf("Masukkan resource yang diminta (3 angka): ");
    for (int i = 0; i < R; i++)
        scanf("%d", &req[i]);
    requestResource(p, req);
    printf("\nAvailable setelah request: [%d, %d, %d]\n",
        available[0], available[1], available[2]);
    return 0;
}

```

Penjelasan:

- $need[i][j] = max[i][j] - allocation[i][j];$
ini untuk menghitung kebutuhan sisa tiap proses.
- $work[j] = available[j];$
ini untuk menyalin resource yang tersedia untuk simulasi pengecekan keamanan.
- $if (need[i][j] > work[j])$
ini untuk mengecek apakah proses bisa dijalankan dengan resource yang ada
- $work[j] += allocation[i][j];$
ini jika resource telah selesai maka resource tersebut akan dikembalikan ke sistem.

- Pretend Allocation (alokasi sementara request)
Sistem seolah-olah memberikan resource untuk mengecek apakah aman atau tidak.
- Rollback jika unsafe
Jika sistem tidak aman, semua perubahan yang dilakukan akan dibatalakan untuk mencegah terjadinya deadlock
- if (isSafe())
ini adalah request yang hanya terjadi jika sistem tetap aman.

Outputnya:

```
agilnggara@RagilAnggara:~/tugas6/bankers$ gcc bankers.c -o bankers
agilnggara@RagilAnggara:~/tugas6/bankers$ ./bankers
=== Banker's Algorithm ===
Available awal: [5, 4, 3]

Masukkan nomor proses (0-3): 1
Masukkan resource yang diminta (3 angka): 0 0 1
Request dapat diberikan. Sistem tetap aman.

Available setelah request: [5, 4, 2]
agilnggara@RagilAnggara:~/tugas6/bankers$ |
```

Cara kerja program:

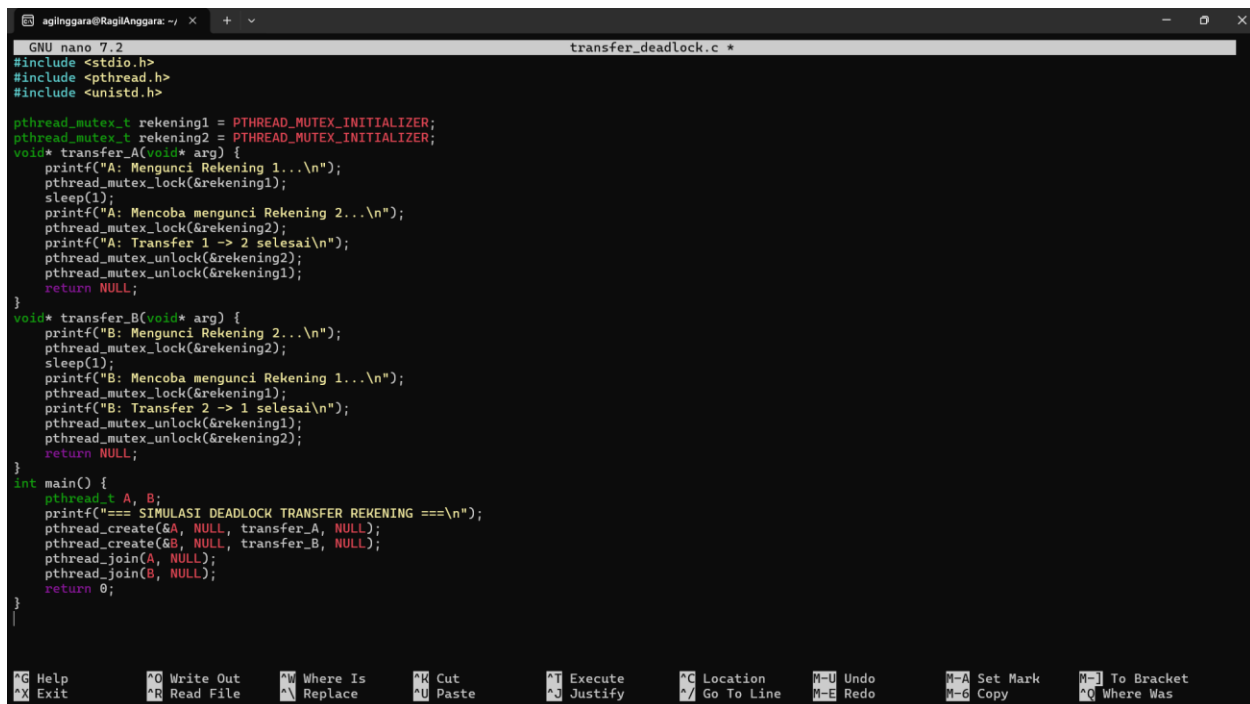
- Need dihitung otomatis $\rightarrow$ Max – Allocation
- User memasukkan:
 - Nomor proses (0–3)
 - Resource yang ingin di-request (3 angka)
- Program:
 - Mengecek apakah request valid
 - Melakukan alokasi sementara
 - Mengecek apakah sistem tetap aman
 - Jika aman maka request disetujui
 - Jika tidak maka rollback dan request ditolak

BAB IV REAL CASE

Buat simulasi deadlock untuk kasus: "Dua orang ingin transfer uang antar rekening secara bersamaan"

- Orang A: transfer dari Rekening 1 ke Rekening 2
- Orang B: transfer dari Rekening 2 ke Rekening 1
- Implementasikan deadlock prevention

Buat file `transfer_deadlock.c`



```
GNU nano 7.2 transfer_deadlock.c
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t rekening1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t rekening2 = PTHREAD_MUTEX_INITIALIZER;
void* transfer_A(void* arg) {
    printf("A: Mengunci Rekening 1...\n");
    pthread_mutex_lock(&rekening1);
    sleep(1);
    printf("A: Mencoba mengunci Rekening 2...\n");
    pthread_mutex_lock(&rekening2);
    printf("A: Transfer 1 -> 2 selesai\n");
    pthread_mutex_unlock(&rekening2);
    pthread_mutex_unlock(&rekening1);
    return NULL;
}
void* transfer_B(void* arg) {
    printf("B: Mengunci Rekening 2...\n");
    pthread_mutex_lock(&rekening2);
    sleep(1);
    printf("B: Mencoba mengunci Rekening 1...\n");
    pthread_mutex_lock(&rekening1);
    printf("B: Transfer 2 -> 1 selesai\n");
    pthread_mutex_unlock(&rekening1);
    pthread_mutex_unlock(&rekening2);
    return NULL;
}
int main() {
    pthread_t A, B;
    printf("=== SIMULASI DEADLOCK TRANSFER REKENING ===\n");
    pthread_create(&A, NULL, transfer_A, NULL);
    pthread_create(&B, NULL, transfer_B, NULL);
    pthread_join(A, NULL);
    pthread_join(B, NULL);
    return 0;
}
```

Penjelasan:

- `pthread_mutex_t rekening1, rekening2`
ini untuk 2 mutex mewakili 2 rekening yang harus di lock sebelum transfer.
- `pthread_mutex_lock(&rekening1) (A)`
Ini untuk A mengunci rekening 1 terlebih dahulu.
- `pthread_mutex_lock(&rekening2) (A)`
ini untuk A mencoba mengunci rekening 2, tetapi akan gagal karena B sudah memegangnya.
- `pthread_mutex_lock(&rekening2) (B)`

ini adalah B mengunci rekening 2 terlebih dahulu.

- `pthread_mutex_lock(&rekening1)` (B)

ini untuk B mencoba mengunci rekening 1, tetapi gagal karena A sudah memegangnya.

- Hasil

A menunggu B dan B menunggu A (Deadlock)

Outputnya:

```
agilnggara@RagilAnggara:~/tugas6/transfer_deadlock$ gcc transfer_deadlock.c -o transfer_deadlock
agilnggara@RagilAnggara:~/tugas6/transfer_deadlock$ ./transfer_deadlock
=== SIMULASI DEADLOCK TRANSFER REKENING ===
A: Mengunci Rekening 1...
B: Mengunci Rekening 2...
A: Mencoba mengunci Rekening 2...
B: Mencoba mengunci Rekening 1...
|
```

Cara Kerja Program:

- A mengunci Rekening 1
- B mengunci Rekening 2
- A butuh Rek. 2 yang dikunci B
- B butuh Rek. 1 yang dikunci A

Jadi yang terjadi adalah kondisi terus saling menunggu dan terjadilah deadlock.

Buat file `transfer_prevention_deadlock.c`

```
GNU nano 7.2 transfer_prevention_deadlock.c
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t rekening[2] = {
    PTHREAD_MUTEX_INITIALIZER,
    PTHREAD_MUTEX_INITIALIZER
};

void lock_ordered(int from, int to) {
    int first = from < to ? from : to;
    int second = from < to ? to : from;
    pthread_mutex_lock(&rekening[first]);
    pthread_mutex_lock(&rekening[second]);
}

void unlock_ordered(int from, int to) {
    int first = from < to ? from : to;
    int second = from < to ? to : from;
    pthread_mutex_unlock(&rekening[second]);
    pthread_mutex_unlock(&rekening[first]);
}

void* transfer_A(void* arg) {
    printf("A: Transfer 1 -> 2, mengunci berdasarkan urutan...\n");
    lock_ordered(0, 1);
    printf("A: Transfer selesai.\n");
    sleep(1);
    unlock_ordered(0, 1);
    return NULL;
}

void* transfer_B(void* arg) {
    printf("B: Transfer 2 -> 1, mengunci berdasarkan urutan...\n");
    lock_ordered(1, 0);
    printf("B: Transfer selesai.\n");
    sleep(1);
    unlock_ordered(1, 0);
    return NULL;
}

int main() {
    pthread_t A, B;
    printf("=== PREVENT DEADLOCK: LOCK ORDERING ===\n");
    pthread_create(&A, NULL, transfer_A, NULL);
    pthread_create(&B, NULL, transfer_B, NULL);
    pthread_join(A, NULL);
    pthread_join(B, NULL);
    return 0;
}
```

```
pthread_create(&B, NULL, transfer_B, NULL);
pthread_join(A, NULL);
pthread_join(B, NULL);
printf("Program selesai tanpa deadlock.\n");
return 0;
}
```

Help Exit Write Out Read File Where Is Replace Cut Paste Execute Justify Location Go To Line Undo Redo Set Mark Copy To Bracket Where Was

Penjelasan:

- `lock_ordered(from, to)`
ini adalah fungsi menentukan urutan penguncian berdasarkan ID rekening.
- `first = from < to ? from : to`
ini untuk menentukan rekening mana yang lebih kecil ID nya.
- `pthread_mutex_lock(&rekening[first])`
ini agar selalu mengunci rekening nomor kecil lebih dahulu.
- `pthread_mutex_lock(&rekening[second])`
ini agar mengunci nomor besar.
- `unlock_ordered()`
ini untuk melepas lock dengan urutan terbalik.
- Hasil
Yang terjadi adalah semua thread mengunci rekening dengan urutan yang sama dan deadlock dicegah.

Outputnya:

```
agilnggara@RagilAnggara:~/tugas6/transfer_prevention_deadlock$ gcc transfer_prevention_deadlock.c -o transfer_prevention_deadlock
agilnggara@RagilAnggara:~/tugas6/transfer_prevention_deadlock$ ./transfer_prevention_deadlock
=== PREVENT DEADLOCK: LOCK ORDERING ===
A: Transfer 1 -> 2, mengunci berdasarkan urutan...
A: Transfer selesai.
B: Transfer 2 -> 1, mengunci berdasarkan urutan...
B: Transfer selesai.
Program selesai tanpa deadlock.
agilnggara@RagilAnggara:~/tugas6/transfer_prevention_deadlock$
```

Cara kerja program:

- Setiap rekening harus dikunci sebelum dipakai, agar tidak dipakai 2 orang sekaligus
- Program selalu mengunci rekening dengan urutan yang sama, rekening bernomor kecil dulu, kemudian rekening bernomor besar
- Ketika A dan B mentransfer bersamaan:
A : Rekening 1 ke 2
B : Rekening 2 ke 1
Jadi keduanya akan tetap mengikuti aturan yang sama, yaitu kunci rekening 1 dulu lalu rekening 2
- Akibatnya
Hanya 1 orang yang bisa mendapatkan 2 kunci. yang lainnya otomatis menunggu sebentar, tidak selamanya dan kemudian deadlock tidak akan terjadi.

